

DEVOPS MASTER COURSE

Volume 7D

Azure Containers, Serverless & Integration

Chapters 31-40

ACR | AKS | CONTAINER APPS | APP SERVICE | FUNCTIONS | API MANAGEMENT |
SERVICE BUS | EVENT GRID | LOGIC APPS

Production-grade cloud-native applications, Kubernetes, serverless compute, APIs, messaging, events, workflows, labs, and enterprise architecture

2026 Edition

About This Part

Volume 7D develops the application platform layer of Azure. Parts A through C established identity, governance, infrastructure, networking, storage, databases, analytics, and migration. This part turns those foundations into secure delivery platforms for containers, Kubernetes, web applications, functions, APIs, messaging, event distribution, and orchestrated workflows.

The engineering objective is not to deploy a demo that responds once. It is to create repeatable application platforms with immutable artifacts, controlled identity, private connectivity, observable releases, back-pressure, idempotency, dead-letter handling, rollback, tested recovery, and explicit ownership. Every service is evaluated as part of a system rather than as an isolated product.

Safety, Security, and Cost Controls

- Use a sandbox subscription, approved regions, budgets, owner tags, environment tags, and expiry dates before provisioning paid services.
- Use Microsoft Entra identities, managed identities, and AKS Workload ID. Do not place registry passwords, publish profiles, connection strings, SAS tokens, or service-principal secrets in code or pipeline logs.
- Prefer private endpoints, private DNS, internal ingress, network controls, least-privilege Azure RBAC, and application authorization appropriate to the data classification.
- Pin images by digest for controlled releases, scan dependencies and images, produce software bills of materials, sign release evidence, and separate build identities from runtime identities.
- Treat at-least-once delivery as normal. Implement idempotency, bounded retries, timeouts, poison-message handling, dead-letter ownership, replay controls, and business reconciliation.
- Use synthetic data and non-production endpoints. Delete idle clusters, Premium registries, API Management instances, dedicated plans, private endpoints, messaging capacity, and log workspaces after labs.

Table of Contents

Chapter 31 Azure Container Registry and Container Supply-Chain Engineering

Chapter 32 Azure Kubernetes Service Production Engineering

Chapter 33 Azure Container Apps and Azure Container Instances

Chapter 34 Azure App Service Production Engineering

Chapter 35 Azure Functions and Serverless Compute Engineering

Chapter 36 Azure API Management and API Platform Engineering

Chapter 37 Azure Service Bus Messaging Engineering

Chapter 38 Azure Event Grid and Event Distribution

Chapter 39 Azure Logic Apps and Durable Workflow Engineering

Chapter 40 Enterprise Cloud-Native Application Architecture and Capstone

Navigation: Heading styles support Word and LibreOffice document-outline navigation and a field-based table of contents.

Chapter 31 - Azure Container Registry and Container Supply-Chain Engineering

Building a trusted artifact system for images, OCI artifacts, regional distribution, promotion, and controlled runtime consumption

Learning Objectives

- Design registry topology, SKU selection, private connectivity, geo-replication, and retention for production workloads.
- Build immutable image-promotion workflows using digests, repository permissions, scanning, provenance, and release evidence.
- Use managed identities and workload identities instead of registry administrator credentials.
- Troubleshoot authentication, DNS, firewall, throttling, image-pull, replication, and stale-tag failures.

31.1 A Registry Is a Production Control Point

Azure Container Registry (ACR) stores private container images and other OCI artifacts, but its operational role is broader than storage. It is the boundary between build systems and runtime platforms. A weak registry design allows mutable tags, unknown base images, broad push permissions, and uncontrolled public dependencies to enter production.

- Create separate repositories or registries according to trust boundaries, ownership, compliance, and blast radius - not merely according to team preference.
- Use one-way promotion. Build once, identify the artifact by digest, and promote the same digest through development, test, staging, and production.
- Do not allow production clusters to pull directly from arbitrary public registries. Import or cache approved public artifacts through ACR and retain evidence of origin.
- Use retention and purge policies carefully. Never delete artifacts still referenced by active deployments, rollback manifests, or forensic evidence.

31.2 Identity, Networking, and Regional Design

Runtime platforms should authenticate through managed identities or Workload ID and receive only pull permissions. Build systems receive scoped push access. Human administrators use Entra authentication and privileged role activation. The registry administrator account is unsuitable for normal operations because it creates shared long-lived credentials.

Decision	Production Guidance
SKU	Use Standard for ordinary private registries; use Premium when private endpoints, geo-replication, connected registry, higher throughput, or advanced network controls are required.
Network	Prefer Private Link and private DNS for internal workloads. Document build-agent and on-premises paths before disabling public access.
Regions	Place replicas close to deployment regions where latency, resilience, and image-pull throughput justify Premium geo-replication.

Decision	Production Guidance
Availability	Registry resilience does not remove the need for pinned digests, local node caches, deployment retries, and rollback artifacts.

Private endpoint design must include both registry and data endpoints, private DNS resolution, pipeline-agent access, and emergency procedures. A firewall rule that permits a build agent but not the data endpoint can produce authentication success followed by layer-download failure.

31.3 Immutable Promotion and Supply-Chain Evidence

1. Build in an isolated, reproducible pipeline using a reviewed Dockerfile and pinned base-image digest.
2. Run source, dependency, secret, infrastructure, and image scans; record exceptions with owners and expiry dates.
3. Generate an SBOM and provenance record. Store evidence with the immutable release artifact.
4. Push once, capture the image digest, and use the digest in deployment manifests or a signed release bundle.
5. Promote by registry-side import or metadata approval rather than rebuilding for each environment.
6. Deploy with a runtime identity that has pull-only access and verify the admitted digest.

```
az acr create -g rg-platform -n acrcontoso001 --sku Premium
az acr import -n acrcontoso001 \
  --source docker.io/library/nginx:1.27 \
  --image approved/nginx:1.27
az acr repository show-tags -n acrcontoso001 \
  --repository approved/nginx --detail
```

Tags remain useful human labels, but they are not immutable identities. Release records should name the digest, source commit, pipeline run, policy result, deployment environment, and rollback digest.

31.4 Operating the Registry

Monitor authentication failures, request rates, storage growth, webhook failures, replication status, image-pull latency, and denied network flows. Establish repository ownership, naming conventions, quarantine procedures, vulnerability-remediation service levels, and an emergency process for revoking a compromised artifact.

- Use artifact cache or controlled import to reduce public-registry rate-limit and availability risk.
- Apply tag locking or repository controls only after validating deployment and rollback workflows.
- Periodically prove that every production image can be traced to a reviewed source revision and scan result.
- Test a registry access outage and confirm that running workloads continue while new deployments fail safely and visibly.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
AKS ImagePullBackOff	Workload identity or kubelet identity lacks AcrPull; image path or digest is wrong	Inspect pod events, image reference, ACR role assignments, identity client ID, registry audit logs, and DNS. Correct scope and redeploy without using admin credentials.

Symptom	Likely Cause	Evidence and Corrective Action
Login succeeds but layers fail	Private DNS or data endpoint path is incomplete	Resolve registry and data endpoints from the build/runtime subnet, inspect Private Link state and firewall logs, then correct DNS links or routes.
Old image runs after release	Mutable tag or node cache reused an earlier manifest	Compare running imageID digest with release evidence, deploy by digest, and prohibit mutable production tags.
Pulls are slow in another region	Registry is distant, throttled, or replication is incomplete	Check request metrics and replica status; add justified geo-replication, tune concurrency, and avoid mass simultaneous cold pulls.

Hands-on Lab 31: Create a Private Registry and Prove Digest-Based Promotion

Prerequisites

- Azure CLI authenticated to a sandbox subscription.
- A VNet/subnet and private DNS permissions if testing Private Link.
- A small container image and an approved cleanup window.

Implementation

1. Create an ACR with a supported training SKU; document which Premium-only controls would be required in production.
2. Disable or avoid the registry administrator account and assign your build identity push access and runtime identity pull access.
3. Build or import a small image, capture its digest, and record source and scan evidence.
4. Create development and production repository paths and promote the same digest without rebuilding.
5. Where budget permits, create a private endpoint and validate name resolution from an authorized subnet.
6. Attempt an unauthorized push and verify denial.
7. Deploy or pull the image by digest and record the resolved digest.

Validation Evidence

- Release evidence ties source revision, scan result, repository, tag, and digest together.
- Runtime identity can pull but cannot push.
- Unauthorized identity and unauthorized network path are denied.
- Development and production references resolve to the same promoted digest.

Cleanup

- Delete test repositories, private endpoints, DNS records, role assignments, and registry if it is not retained for later labs.
- Confirm no Premium replica or private endpoint remains billable.

Interview Questions and Model Answers

1. Why is a digest stronger than a tag for production deployment?

A digest identifies exact content. A tag is a mutable pointer and may be moved or cached, making rollback and evidence ambiguous.

2. When is ACR Premium justified?

When private endpoints, geo-replication, connected registry, advanced network controls, or higher throughput are required and their cost is supported by workload objectives.

3. Why should production clusters not use the ACR admin account?

It is a shared long-lived credential with broad access. Managed identities or Workload ID provide scoped, auditable, automatically managed authentication.

4. How do you reduce dependence on public registries?

Import or cache approved artifacts in ACR, pin digests, scan them, retain provenance, and make runtime environments pull only from controlled registries.

Chapter Summary

- Treat ACR as a trust boundary and release control point.
- Build once, promote immutable digests, and retain supply-chain evidence.
- Use managed identity, private connectivity, and least privilege.
- Operate registry capacity, replication, retention, and incident response as production services.

Chapter 32 - Azure Kubernetes Service Production Engineering

Operating managed Kubernetes with secure identity, networking, scaling, upgrades, policy, observability, and platform ownership

Learning Objectives

- Design AKS clusters, node pools, networking, ingress, private access, and identity boundaries.
- Use Microsoft Entra Workload ID, Azure RBAC, Kubernetes RBAC, and managed identities correctly.
- Engineer autoscaling, disruption controls, upgrades, backups, policy, and observability for production.
- Diagnose scheduling, networking, DNS, storage, identity, image, and control-plane access failures.

32.1 Managed Kubernetes Still Requires a Platform Team

AKS manages the Kubernetes control plane and integrates Azure services, but the customer still owns workload architecture, node pools, Kubernetes objects, network policy, identity, resource requests, upgrades, admission controls, data protection, and incident response. A production AKS service therefore needs a defined platform product, not an unmanaged shared cluster.

Boundary	Key Ownership
Azure managed	Control-plane service operation, supported version images, managed add-ons, platform SLA according to configuration.
Platform team	Cluster lifecycle, node pools, networking, policy, identity, ingress, observability, upgrade waves, capacity, backup, and developer interfaces.
Application team	Deployments, requests/limits, probes, disruption budgets, secrets usage, telemetry, data semantics, and rollback.
Security/SRE	Guardrails, vulnerability response, threat monitoring, SLOs, game days, and incident command.

32.2 Cluster, Node-Pool, and Network Design

Use system node pools for critical add-ons and separate user node pools by workload characteristics, trust, architecture, operating system, GPU need, or disruption tolerance. Availability zones, multiple replicas, topology spread constraints, anti-affinity, and PodDisruptionBudgets reduce correlated failure, but only when resource capacity exists to reschedule workloads.

- Choose Azure CNI mode and network architecture from IP capacity, network-policy, observability, and connectivity requirements. Reserve address space for growth and upgrades.
- Use private clusters or API server VNet integration where operational access paths, DNS, and incident response are designed in advance.
- Expose applications through a controlled ingress or Gateway API implementation. Terminate TLS with managed certificate processes and restrict internal services.
- Use network policies and Azure perimeter controls together. Kubernetes policy does not replace NSGs, firewall, private endpoints, or application authorization.

32.3 Identity and Workload Access

Microsoft Entra Workload ID maps a Kubernetes service account to an Azure identity using projected service-account tokens and federated credentials. It replaces deprecated pod-identity approaches and avoids

embedding client secrets. Each workload receives a dedicated service account and the minimum Azure role assignment at the narrowest practical scope.

```
az aks create -g rg-aks -n aks-prod \
--enable-oidc-issuer --enable-workload-identity \
--enable-managed-identity --node-count 3

az aks show -g rg-aks -n aks-prod \
--query '{oidc:oidcIssuerProfile.issuerUrl,wi:securityProfile.workloadIdentity}'
```

Cluster administrator access, Kubernetes authorization, Azure resource access, and container-registry pull access are separate decisions. Troubleshooting must identify which identity and which authorization layer denied the request.

32.4 Scaling, Upgrades, Reliability, and Operations

Horizontal Pod Autoscaler scales replicas from metrics; KEDA scales from event sources; Vertical Pod Autoscaler can recommend or adjust requests; cluster autoscaler changes node capacity. These mechanisms interact. Incorrect requests, missing metrics, restrictive disruption budgets, unavailable IP addresses, or node-pool limits can prevent safe scaling.

- Use supported Kubernetes versions, automatic or controlled upgrade channels, maintenance windows, preproduction rings, surge capacity, and workload compatibility tests.
- Configure requests, limits, startup/readiness/liveness probes, PodDisruptionBudgets, topology spread, graceful termination, and deployment rollback.
- Collect control-plane logs, Container Insights or managed Prometheus metrics, application traces, audit data, and Kubernetes events with retention appropriate to incident needs.
- Back up required cluster state and persistent data; regularly rebuild a cluster from code to prove that configuration is not trapped in the control plane.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Pods remain Pending	Insufficient CPU/memory/IPs, taints, selectors, affinity, PVC, quota, or autoscaler limits	Use <code>kubectl describe pod</code> , scheduler events, node allocatable resources, subnet utilization, node-pool limits, and PVC status; correct the scheduling constraint rather than forcing a restart.
Workload cannot reach Key Vault	Federated credential, service account, audience, role, DNS, or network path is wrong	Inspect service-account annotation, projected token, OIDC issuer, identity role, private DNS, and firewall logs.
Upgrade stalls	PDB, surge capacity, quota, unhealthy nodes, or incompatible API objects	Review upgrade operation, PDBs, deprecated APIs, quota, subnet capacity, and drain events; add capacity or remediate workloads before retrying.
Service is reachable intermittently	Probe failure, topology imbalance, ingress timeout, SNAT, or unhealthy endpoints	Correlate ingress logs, endpoints, pod restarts, readiness, node placement, network flows, and application latency.

Hands-on Lab 32: Deploy a Secure AKS Workload with Workload ID and Autoscaling

Prerequisites

- A sandbox subscription with quota for a small AKS cluster.
- ACR image from Chapter 31 and an Azure identity with scoped test access.
- kubectl, Helm if required, and a cleanup budget.

Implementation

1. Create or use a small AKS cluster with managed identity, OIDC issuer, and Workload ID enabled.
2. Create separate system and user node-pool intent in the design; use a small practical configuration for the lab.
3. Assign AcrPull to the kubelet identity and deploy an image by digest.
4. Create a workload identity, federated credential, Kubernetes service account, and scoped Azure role assignment.
5. Deploy an application with requests, limits, probes, PodDisruptionBudget, and topology-spread configuration.
6. Configure HPA or KEDA for a measurable load source and run a controlled load test.
7. Perform a rolling update and verify rollback to the prior digest.

Validation Evidence

- Pod imageID matches the approved ACR digest.
- Workload accesses the authorized Azure resource without a stored secret and is denied outside its role.
- Scaling reacts to load and returns to the configured minimum.
- A rollout and rollback preserve availability within the stated test objective.

Cleanup

- Delete workload role assignments, identities, federated credentials, test namespaces, and cluster if not reused.
- Confirm load-test and monitoring resources are stopped or removed.

Interview Questions and Model Answers

1. What does AKS manage, and what remains the customer responsibility?

Azure manages the control-plane service; the customer owns workload configuration, node pools, identity, networking, policy, upgrades, observability, data protection, and operational readiness.

2. Why is Workload ID preferred over a client secret?

It exchanges projected Kubernetes tokens for Entra tokens through federation, eliminating stored credentials and enabling scoped, auditable access.

3. Why can cluster autoscaler fail even when pods are Pending?

The pending pod may violate selectors, taints, affinity, quotas, IP capacity, PVC topology, or maximum-node settings; adding nodes would not necessarily make it schedulable.

4. How do you make AKS upgrades safer?

Use supported versions, preproduction rings, maintenance windows, surge capacity, deprecated-API checks, PDB review, observability, and tested rollback or rebuild procedures.

Chapter Summary

- AKS is a platform product that requires explicit ownership and guardrails.

- Use Workload ID and separate control-plane, Kubernetes, and Azure authorization layers.
- Design scaling, upgrades, disruption, IP capacity, and observability together.
- Prove recovery by rebuilding clusters and restoring state from controlled sources.

Chapter 33 - Azure Container Apps and Azure Container Instances

Choosing serverless container platforms for HTTP services, event-driven workers, scheduled jobs, and isolated short-lived execution

Learning Objectives

- Select Container Apps, Container Apps Jobs, ACI, AKS, or App Service from workload requirements.
- Engineer revisions, traffic splitting, KEDA scaling, Dapr integration, jobs, networking, identity, and secrets.
- Use workload profiles and environments to isolate capacity and control cost.
- Troubleshoot revision, ingress, registry, scaling, job, Dapr, DNS, and network failures.

33.1 Choose the Abstraction Level Deliberately

Azure Container Apps provides a serverless application platform built for containers, revisions, ingress, KEDA-based autoscaling, jobs, and optional Dapr APIs without exposing Kubernetes administration. Azure Container Instances provides direct, isolated container groups for simple or short-lived workloads. AKS is appropriate when teams need Kubernetes APIs, ecosystem control, custom operators, or deeper platform extensibility.

Service	Best Fit	Key Trade-off
Container Apps	Microservices, APIs, event workers, scale-to-zero, revisions, Dapr, managed environments	Less Kubernetes control; platform limits and networking model must fit.
Container Apps Jobs	Manual, scheduled, or event-driven finite tasks	Job semantics rather than long-running service semantics.
ACI	Simple burst tasks, build agents, isolated utilities, quick container groups	Limited orchestration and application-platform capabilities.
AKS	Kubernetes platform, operators, complex networking, broad ecosystem	Highest operational responsibility.

33.2 Environments, Revisions, Ingress, and Traffic

A Container Apps environment is the security, networking, and observability boundary for multiple apps and jobs. Revisions are immutable snapshots of revision-scope configuration. Single-revision mode simplifies replacement; multiple-revision mode supports traffic splitting, canary releases, and side-by-side validation.

- Use internal environments or restricted ingress when services should not be public.
- Keep secrets and environment-specific settings outside images. Prefer managed identity and Key Vault integration patterns.
- Use revision labels and explicit traffic weights for progressive delivery; observe errors, latency, and business outcomes before promotion.
- Design egress, private DNS, VNet integration, private endpoints, and downstream firewall rules as one path.

33.3 KEDA Scaling, Jobs, and Dapr

Container Apps uses KEDA-supported scale triggers. HTTP concurrency, queue depth, event-stream lag, CPU, memory, and custom scalers can drive replica count. Scale-to-zero saves cost but introduces cold-start and connection behavior that must fit the SLO. Minimum replicas or workload profiles may be needed for latency-sensitive services.

```
az containerapp update -g rg-app -n orders-api \
--image acrcontoso001.azurecr.io/orders@sha256:<digest>

az containerapp revision list -g rg-app -n orders-api -o table
az containerapp ingress traffic set -g rg-app -n orders-api \
--revision-weight orders--old=90 orders--new=10
```

Dapr can provide service invocation, state, pub/sub, bindings, and secrets abstractions, but it adds runtime behavior and configuration ownership. Use it where the portability and consistency benefit exceeds the added operational surface. Container Apps Jobs are preferable for finite tasks; implement checkpointing and idempotency because retries can repeat work.

33.4 Production Operations and Cost

Observe revision health, replica count, scaler decisions, ingress latency, console/system logs, job executions, Dapr sidecars, and downstream dependencies. Define maximum replicas to protect budgets and dependencies, but ensure limits do not silently cap required throughput. For dedicated workload profiles, track reserved capacity utilization.

- Test cold starts, scale-from-zero, maximum-scale behavior, and dependency saturation.
- Retain failed job logs and payload identifiers for replay while protecting sensitive content.
- Use image digests and ACR managed identity. Avoid registry username/password secrets.
- Delete inactive revisions and abandoned environments only after rollback and forensic retention requirements expire.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
New revision receives no traffic	Traffic weights or label target the old revision; ingress is disabled	List revisions and traffic configuration, verify active state and target port, then update weights deliberately.
App never scales from zero	Scaler authentication, metadata, event source, or KEDA rule is wrong	Inspect system logs, scaler status, managed identity, queue metrics, rule metadata, and network path.
Private dependency fails	VNet integration, private DNS, route, NSG, or firewall mismatch	Resolve the endpoint inside the environment, trace egress route and identity, and inspect downstream deny logs.
Job repeats business action	At-least-once trigger or retry without idempotency	Use execution/message IDs, conditional writes, checkpoints, and reconciliation before replaying.

Hands-on Lab 33: Deploy a Canary Container App and Event-Driven Job

Prerequisites

- A small ACR image by digest.
- A sandbox Container Apps environment and a queue or event source.
- A managed identity with narrowly scoped permissions.

Implementation

1. Create a Container Apps environment and document the production internal/private design.
2. Deploy version 1 of a small HTTP app from ACR by digest with managed identity.
3. Deploy version 2 as a new revision and route 10 percent of traffic to it.
4. Collect response-version, latency, and error evidence, then promote or roll back.
5. Create a finite Container Apps Job with a scheduled or event-driven trigger.
6. Implement an idempotency key and force one retry to prove duplicate safety.
7. Set minimum, maximum, and scaling rules and observe replica or execution behavior.

Validation Evidence

- Traffic weights and revision response evidence match the canary plan.
- The runtime pulls from ACR without registry credentials.
- The job retry does not duplicate the protected business result.
- Scale limits and logs are visible and documented.

Cleanup

- Delete test revisions, jobs, queue contents, identities, and environment if not reused.
- Confirm no dedicated workload profile or Log Analytics workspace remains unnecessarily.

Interview Questions and Model Answers

1. When would you choose Container Apps instead of AKS?

When the workload fits managed revisions, ingress, KEDA scaling, jobs, and optional Dapr without requiring Kubernetes API control, operators, or custom cluster administration.

2. What is the operational value of a revision?

It is an immutable application snapshot that supports controlled activation, traffic splitting, rollback, and comparison of telemetry between versions.

3. Why can scale-to-zero be risky?

Cold starts, connection establishment, dependency initialization, and first-request latency may violate SLOs; minimum replicas may be necessary.

4. When is ACI appropriate?

For simple isolated container groups or short-lived tasks where full application-platform features and orchestration are unnecessary.

Chapter Summary

- Choose the least operationally complex container platform that meets requirements.
- Use revisions and traffic weights for observable progressive delivery.
- Treat KEDA triggers, identity, private networking, and idempotency as one design.
- Use jobs for finite work and ACI for simple isolated execution.

Chapter 34 - Azure App Service Production Engineering

Operating managed web applications and APIs with deployment slots, identity, private connectivity, scaling, configuration, and recovery

Learning Objectives

- Design App Service plans, apps, slots, scaling, availability zones, and regional topology.
- Separate inbound private access from outbound VNet integration and dependency connectivity.
- Use managed identity, Key Vault references, authentication, TLS, and configuration controls.
- Troubleshoot startup, deployment, DNS, SNAT, scaling, slot swap, certificate, and dependency failures.

34.1 App Service as a Managed Application Platform

Azure App Service runs web applications and APIs without exposing VM administration. The App Service plan defines regional compute, operating system, scaling boundary, and cost. Multiple apps can share a plan, which improves utilization but also creates noisy-neighbor, scaling, and blast-radius considerations.

- Separate production from non-production plans and isolate workloads with different scaling, compliance, or failure characteristics.
- Use deployment slots on supported tiers to stage releases, warm dependencies, validate health, and swap configuration safely.
- Use zone-redundant plans where supported and justified, but design regional recovery independently.
- Keep local file-system assumptions minimal. Store durable state in managed data services.

34.2 Inbound, Outbound, DNS, and Identity

Private endpoints control inbound access to an app. VNet integration controls outbound access from the app to resources in or through a virtual network. They solve different directions and are often both required. Access restrictions provide ordered inbound allow/deny rules for public endpoints but do not turn the app into a private endpoint.

Requirement	Primary Mechanism
Private inbound application endpoint	App Service private endpoint plus private DNS and disabled/restricted public access.
Private outbound dependency access	Regional VNet integration, correct routes/DNS, and private endpoints or service endpoints on dependencies.
Stable outbound public IP	VNet integration with route-all design and NAT Gateway where supported/appropriate.
Secretless dependency authentication	System- or user-assigned managed identity with scoped roles; Key Vault references for unavoidable secrets.

34.3 Deployment Slots and Safe Releases

Deploy code to a staging slot, apply slot-specific settings, warm the application, execute synthetic tests, and then swap. A swap changes routing and selected configuration, not merely files. Incorrectly marked settings can move production connection strings or feature flags into the wrong slot.

```
az webapp deployment slot create -g rg-web -n contoso-api \
--slot staging
az webapp config appsettings set -g rg-web -n contoso-api \
--slot staging --settings RELEASE_RING=staging
az webapp deployment slot swap -g rg-web -n contoso-api \
--slot staging --target-slot production
```

Use health checks, warm-up behavior, compatible database changes, telemetry comparison, and a clear swap-back decision. A rollback is not safe when the new version performed irreversible data changes.

34.4 Scaling, Observability, and Recovery

Plan scaling from CPU, memory, requests, queue length, or schedule while respecting downstream capacity. Monitor HTTP status distribution, response time, worker restarts, memory, connections, SNAT, deployment events, health-check status, and dependency telemetry. Use Application Insights/OpenTelemetry and centralized logs rather than relying only on live log streaming.

- Test scale-out and scale-in with session state externalized and connection pools bounded.
- Back up or redeploy application content from source; protect databases and storage using their native recovery mechanisms.
- Use custom domains and certificates with automated renewal where possible; monitor expiry and binding state.
- Document regional recreation, DNS or Front Door failover, configuration restoration, and identity reauthorization.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
App returns 502/503 after deploy	Startup crash, wrong port, unhealthy worker, or dependency failure	Inspect deployment logs, startup command, App Service diagnostics, health check, application logs, and dependency telemetry; roll back slot if needed.
Private database cannot be reached	VNet integration, DNS, route, NSG, or database firewall issue	Resolve private FQDN from the app, inspect VNet integration and route-all, test TCP reachability, and review downstream firewall logs.
Slot swap changes wrong settings	Connection strings or app settings are not marked slot-specific	Compare slot configuration before swap, mark environment-specific values as deployment-slot settings, and perform swap preview.
Intermittent outbound failures	SNAT exhaustion, connection leak, DNS churn, or downstream throttling	Inspect connection metrics and dependency failures; reuse connections, bound pools, add NAT design if justified, and implement retry with jitter.

Hands-on Lab 34: Deploy an App Service API with Private Dependency Access and Slot Swap

Prerequisites

- A sandbox resource group and supported App Service plan.
- A small web API package or container.
- A Key Vault or test dependency and managed identity permissions.

Implementation

1. Create an App Service plan and application with a system-assigned managed identity.
2. Configure a test dependency through managed identity or Key Vault reference rather than a plaintext secret.
3. Create a staging slot and mark environment-specific settings as slot settings.
4. Deploy version 1 to production and version 2 to staging.
5. Enable health checks, run synthetic validation, and perform a controlled swap.
6. Where feasible, configure VNet integration and prove private DNS/dependency access.
7. Swap back or redeploy the prior artifact and record rollback evidence.

Validation Evidence

- No secret appears in source, deployment package, or app settings export.
- Staging and production maintain correct slot-specific configuration.
- Health and telemetry evidence supports the swap decision.
- Rollback restores the prior version without losing environment configuration.

Cleanup

- Delete slots, app, plan, VNet integration resources, test secrets, and role assignments if not reused.
- Confirm custom domains or certificates were not left in an unintended state.

Interview Questions and Model Answers

1. What is the difference between a private endpoint and VNet integration for App Service?

A private endpoint provides private inbound access to the app. VNet integration provides outbound access from the app to resources in or through a VNet.

2. Why are deployment slots safer than deploying directly to production?

They allow warm-up, validation, telemetry comparison, slot-specific configuration, controlled swap, and rapid swap-back.

3. What creates App Service noisy-neighbor risk?

Multiple apps share the same plan capacity and scaling boundary; one app can consume CPU, memory, connections, or workers needed by another.

4. How do you reduce SNAT-related failures?

Reuse connections, bound pools, avoid per-request clients, monitor outbound connections, use private endpoints where appropriate, and design stable egress with NAT when justified.

Chapter Summary

- App Service removes VM operations but not release, identity, networking, scaling, or recovery engineering.
- Understand inbound private endpoints and outbound VNet integration as separate paths.
- Use deployment slots with health, telemetry, compatible data changes, and swap controls.
- Externalize state and prove scaling and regional recreation.

Chapter 35 - Azure Functions and Serverless Compute Engineering

Designing event-driven functions with Flex Consumption, concurrency, triggers, identity, idempotency, observability, and durable state

Learning Objectives

- Choose Functions hosting and scaling options from latency, networking, runtime, concurrency, and cost needs.
- Implement trigger-safe idempotency, retries, poison handling, timeouts, and bounded downstream pressure.
- Use managed identity, private connectivity, Key Vault, and least-privilege bindings.
- Operate Flex Consumption, dedicated plans, and Durable Functions with observable deployment and recovery.

35.1 Functions Are Distributed Systems, Not Small Scripts

Azure Functions executes code in response to HTTP requests, schedules, queues, streams, database changes, and other events. The programming model is simple, but production behavior includes at-least-once delivery, concurrency, scaling, retries, partial failure, cold starts, and dependency limits. Function code must therefore be designed like any other distributed service.

Hosting Option	Use When	Engineering Focus
Flex Consumption	Modern serverless scale, VNet integration, configurable memory, always-ready instances, event-driven workloads	Cold start, per-function scaling groups, concurrency, cost, supported features.
Premium	Predictable warm capacity, advanced networking, longer or demanding workloads	Reserved capacity, scaling floor, plan utilization.
Dedicated/App Service plan	Existing plan capacity or steady predictable workload	Capacity isolation, scaling ownership, idle cost.
Container Apps	Customized containerized Functions alongside container microservices	Container lifecycle, environment and revision operations.

35.2 Trigger Semantics, Idempotency, and Back-Pressure

A trigger acknowledges work only after the runtime and extension-specific completion rules are met. Retries, lease recovery, duplicate delivery, and host restarts can cause repeated execution. Exactly-once business outcome is achieved with application design, not by assuming exactly-once invocation.

- Use a stable event or business key and a conditional write, inbox table, or state transition to reject duplicates.
- Keep retry counts bounded and use exponential backoff with jitter. Move poison events to a dead-letter destination with ownership and alerting.
- Control concurrency and batch size so a scaled function does not overwhelm databases, APIs, or third-party systems.
- Use checkpoints and compensating workflows for long processing. Avoid holding an HTTP request open for asynchronous business processes.

35.3 Flex Consumption, Deployment, and Identity

Flex Consumption is the recommended consumption-based hosting model for new supported applications. It provides configurable instance memory, per-function scaling behavior, always-ready instances, VNet integration, and deployment controls. Existing classic Consumption applications should be evaluated for migration rather than copied blindly.

```
az functionapp create -g rg-func -n fn-orders-001 \
--storage-account stfunc001 \
--flexconsumption-location <region> \
--runtime python --runtime-version 3.12

az functionapp identity assign -g rg-func -n fn-orders-001
```

Use managed identity for Service Bus, Storage, Key Vault, databases, and APIs where supported. Configuration references should not expose secrets to code. Network integration, private DNS, and dependency firewalls must be validated from the running function environment.

35.4 Durable Functions and Observability

Durable Functions adds stateful orchestrator, activity, and entity patterns. Orchestrator code must be deterministic because execution history is replayed. External I/O, current time, random values, and nondeterministic libraries belong in activities or framework-provided deterministic APIs.

- Use orchestration instance IDs as business correlation and idempotency boundaries.
- Choose the supported Durable backend and retention design deliberately; monitor history growth and stuck instances.
- Emit structured logs with invocation, trace, event, and business IDs. Correlate trigger latency, execution duration, failure, retry, and dependency telemetry.
- Test host restart, duplicate event, poison event, dependency throttling, and long-running orchestration recovery.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Function does not trigger	Connection/identity, extension, filter, network, or disabled trigger issue	Inspect host logs, trigger source metrics, role assignments, extension version, private DNS, firewall, and poison/dead-letter destination.
Downstream database collapses during burst	Unbounded concurrency or connection creation	Measure active executions and connections; reduce concurrency/batch size, reuse clients, add queue buffering, and scale the dependency deliberately.
Durable orchestration behaves inconsistently	Nondeterministic orchestrator code or incompatible deployment	Inspect replay logs and history; move I/O/random/time to activities or deterministic APIs and use version-safe orchestration patterns.
High first-request latency	Scale-from-zero, package size, dependency initialization, or DNS/TLS setup	Measure cold-start phases, reduce package/init work, use always-ready instances or a different hosting plan when the SLO requires it.

Hands-on Lab 35: Build an Idempotent Queue-Triggered Function and Durable Workflow

Prerequisites

- A Functions-supported region and storage account.
- A Service Bus queue or Storage queue with test messages.
- Application Insights or an approved telemetry destination.

Implementation

1. Create a Function App using an appropriate low-cost hosting option and enable managed identity.
2. Grant receive rights to a queue without storing a connection string where supported.
3. Implement a queue-triggered function that uses a stable message ID and conditional idempotency record.
4. Configure bounded retries and a dead-letter/poison path.
5. Send duplicate messages and prove one business result.
6. Implement a small Durable orchestration with two activities and one compensation or failure branch.
7. Force a transient dependency failure and capture retry, trace, and final-state evidence.

Validation Evidence

- Duplicate invocations do not duplicate the business outcome.
- Poison work reaches the owned failure path with useful correlation data.
- Durable orchestration survives replay and records a deterministic final state.
- Telemetry connects source event, function invocation, dependency call, and business record.

Cleanup

- Delete function app, hosting capacity, queue, storage artifacts, idempotency records, and role assignments if not reused.
- Confirm telemetry retention and private endpoints do not create unintended ongoing cost.

Interview Questions and Model Answers

1. Why does at-least-once delivery require idempotency?

A trigger can redeliver after retries, lease recovery, host failure, or ambiguous acknowledgments. The application must prevent repeated business effects.

2. When would you use Flex Consumption instead of Premium?

Use Flex for modern consumption-based scaling and VNet integration when cold-start and feature behavior fit; use Premium for reserved warm capacity or requirements needing more predictable always-on performance.

3. Why must Durable orchestrator code be deterministic?

The runtime replays history to rebuild state. Nondeterministic operations would produce different decisions during replay.

4. How do you protect a database from serverless scaling?

Control concurrency and batch size, reuse connections, queue work, implement back-pressure and retries, and cap function scale according to dependency capacity.

Chapter Summary

- Treat every function as a distributed-system participant.

- Use idempotency, bounded retries, poison handling, and downstream back-pressure.
- Prefer managed identity and validate private network paths.
- Use deterministic Durable workflows and end-to-end correlation telemetry.

Chapter 36 - Azure API Management and API Platform Engineering

Governing APIs with gateways, policies, products, identity, versioning, observability, hybrid deployment, and safe lifecycle management

Learning Objectives

- Design API Management topology, gateway placement, tiers, networking, and multi-region or self-hosted patterns.
- Apply policies for authentication, authorization, transformation, validation, throttling, caching, and resilience safely.
- Manage API versions, revisions, products, subscriptions, developer onboarding, and deprecation.
- Troubleshoot gateway, policy, certificate, DNS, backend, quota, and latency failures.

36.1 API Management Is a Governance and Runtime Layer

Azure API Management (APIM) provides a managed gateway, management plane, developer experiences, policies, analytics, and lifecycle controls. It does not replace backend authorization or application correctness. The gateway should enforce consistent edge concerns while business rules remain in services.

- Define API ownership, data classification, consumers, SLOs, authentication, authorization, quotas, deprecation, and incident contacts before publishing.
- Use products and groups to package access; use subscriptions only where they add governance and do not mistake a subscription key for strong user identity.
- Use revisions for nonbreaking implementation changes and versions for intentional contract changes.
- Keep policy reuse controlled through fragments or reviewed templates and test changes in nonproduction gateways.

36.2 Gateway Topology and Network Design

Every APIM instance includes a managed gateway. Supported tiers and configurations can add multi-region capacity, private networking, or self-hosted gateways placed near hybrid backends. A self-hosted gateway remains managed by the cloud control plane but runs as a container in customer infrastructure, creating additional lifecycle and connectivity responsibilities.

Topology	Use Case	Primary Risk
Public managed gateway	Internet APIs with WAF/Front Door and public backends or controlled private integration	Exposure, certificate and DDoS/WAF design.
Private/internal APIM	Internal APIs and private backends	DNS, private connectivity, operator access, and tier capability.
Multi-region managed gateway	Global API traffic and regional resilience	Cost, backend locality, state, failover, and policy consistency.
Self-hosted gateway	Hybrid or multicloud local API traffic	Gateway container operations, connectivity to APIM, version support, and local observability.

36.3 Policies, Identity, and Backend Protection

Policies execute in ordered scopes and can validate tokens, enforce claims, set headers, transform bodies, limit rate, cache, route, retry, and authenticate to backends using managed identity. Policy editors therefore hold powerful runtime authority and must be governed like code.

```
<policies>
  <inbound>
    <base />
    <validate-jwt header-name="Authorization"
      failed-validation-httpcode="401">
      <openid-config url="https://login.microsoftonline.com/<tenant>/v2.0/.well-known/openid-configuration" />
      <audiences><audience>api://orders</audience></audiences>
    </validate-jwt>
    <rate-limit-by-key calls="60" renewal-period="60"
      counter-key="@(<context.Principal?.Identity?.Name ?? context.Request.IpAddress>)" />
  </inbound>
  <backend><base /></backend>
  <outbound><base /></outbound>
  <on-error><base /></on-error>
</policies>
```

Validate issuer, audience, signature, expiration, and required claims. Do not log tokens or sensitive payloads. When APIM uses managed identity to call a backend, restrict policy-edit permission because a policy author can influence where obtained tokens are sent.

36.4 Reliability, Observability, and API Lifecycle

Collect gateway requests, response codes, latency split, backend time, policy errors, capacity, certificate status, and dependency telemetry. Use correlation headers and trace context. Rate limits protect fairness but are not a complete DDoS strategy. Retries should be limited to safe operations and must not amplify an unhealthy backend.

- Publish explicit error contracts and do not expose backend stack traces.
- Test certificate rotation, backend DNS changes, token-key rollover, quota exhaustion, and regional failure.
- Monitor deprecated versions and consumer usage before retirement. Provide migration windows and evidence of notification.
- Use Infrastructure as Code and API definition sources so gateway configuration can be reviewed, promoted, and rebuilt.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
401/403 at gateway	JWT issuer/audience/claim mismatch, subscription requirement, or APIM authorization policy	Use APIM trace in a protected environment, inspect token metadata without exposing it, compare policy scope and consumer configuration.
502/504 from APIM	Backend DNS, TLS, firewall, timeout, or unhealthy service	Separate gateway and backend latency, test backend from network path, inspect certificates, private DNS, routes, and backend logs.

Symptom	Likely Cause	Evidence and Corrective Action
Policy change breaks unrelated APIs	Policy inherited from global/product scope or fragment change	Review effective policy hierarchy, revision history, deployment diff, and rollback the scoped change.
Latency rises under load	Gateway capacity, policy cost, backend saturation, or connection behavior	Inspect APIM capacity and latency dimensions, simplify expensive transformations, scale gateway tier/capacity, and remediate backend bottleneck.

Hands-on Lab 36: Publish and Protect an API with JWT Validation and Managed Identity

Prerequisites

- A small backend API in App Service, Functions, Container Apps, or AKS.
- An Entra application registration or test identity provider.
- A training APIM instance appropriate to the lab budget.

Implementation

1. Import an OpenAPI definition and create a nonproduction API revision.
2. Configure a backend and use APIM managed identity where the backend supports Entra authorization.
3. Add JWT validation for issuer, audience, and a required claim.
4. Add bounded rate limiting and a standardized error response without logging sensitive tokens.
5. Create a product and test consumer access.
6. Publish a second revision, test it, and make it current without changing the public version.
7. Generate successful, unauthorized, throttled, and backend-failure requests and collect traces.

Validation Evidence

- Unauthorized and incorrectly scoped tokens are rejected.
- APIM reaches the backend without a stored backend secret.
- Rate limiting produces the documented response and does not hide backend failures.
- The revision change is reversible and consumer contract remains stable.

Cleanup

- Delete test subscriptions, products, APIs, identities, and APIM instance if not reused.
- Remove diagnostic settings and certificates created solely for the lab.

Interview Questions and Model Answers

1. What should APIM policies handle versus backend code?

Policies should handle consistent gateway concerns such as token validation, quotas, routing, and transformation; business authorization and domain rules remain in the backend.

2. What is the difference between an API revision and version?

A revision changes an API implementation without intentionally changing the contract; a version represents a distinct public contract consumers may choose.

3. Why is APIM policy-edit permission sensitive?

Policies can transform traffic, authenticate with managed identity, route tokens or payloads, and affect every API in inherited scopes.

4. When is a self-hosted gateway useful?

When APIs are hosted on-premises or across clouds and local traffic, compliance, or latency requires a gateway close to those backends while retaining centralized APIM management.

Chapter Summary

- Use APIM as a governed gateway and lifecycle platform, not as a substitute for backend security.
- Design topology, tier, private networking, certificates, and regional behavior from requirements.
- Treat policies as privileged, reviewed code.
- Operate API contracts, revisions, versions, quotas, and deprecation with consumer evidence.

Chapter 37 - Azure Service Bus Messaging Engineering

Designing reliable queues and topics with delivery semantics, sessions, transactions, duplicate detection, dead-letter ownership, and geo-resilience

Learning Objectives

- Choose queues, topics, subscriptions, sessions, filters, and tiers from business semantics.
- Implement idempotent consumers, locks, retries, duplicate detection, transactions, and dead-letter operations.
- Use managed identity, private endpoints, capacity planning, and observability.
- Distinguish Geo-Disaster Recovery from data-replicating Geo-Replication and design continuity accordingly.

37.1 Broker Semantics Before Entity Creation

Azure Service Bus is an enterprise message broker for decoupled commands, work queues, publish-subscribe distribution, ordering groups, transactions, scheduling, and dead-letter isolation. The design begins with business semantics: who owns the message, how long it is valid, whether order matters, what constitutes duplicate processing, and how failed work is reconciled.

Pattern	Entity Design	Important Semantics
Competing workers	Queue	One active processing outcome per message, lock renewal, idempotency, poison handling.
Publish-subscribe	Topic plus filtered subscriptions	Each matching subscription owns independent delivery, retry, and DLQ state.
Ordered workflow per key	Sessions with SessionId	Order is scoped to a session; throughput and hot-key behavior must be evaluated.
Atomic broker operations	Transactions within supported scope	Broker atomicity does not automatically make external database work atomic.

37.2 Delivery, Locks, Retries, and Idempotency

Peek-lock receives a message without deleting it. The consumer completes it after successful processing, abandons it for retry, defers it for later retrieval, or dead-letters it. Lock loss can occur when processing exceeds the lock and renewal fails. Receive-and-delete is unsuitable where message loss is unacceptable.

- Use stable MessageId and business keys. Duplicate detection reduces repeated sends within a configured window but does not replace idempotent consumers.
- Bound delivery attempts and route poison messages to the DLQ with reason, description, correlation ID, and runbook ownership.
- Use TTL and scheduled enqueue time from business validity, not arbitrary defaults.
- Control prefetch, concurrency, and batch size so locks and downstream capacity remain healthy.

37.3 Identity, Networking, Capacity, and Geo-Resilience

Use managed identities with Azure Service Bus Data Sender or Data Receiver roles at namespace or entity scope. Prefer private endpoints for sensitive internal messaging and validate private DNS. Premium provides dedicated messaging units, predictable isolation, zone-redundancy options, and advanced geo capabilities.

```
az servicebus namespace create -g rg-msg -n sb-contoso-001 \
--sku Premium --location <region>
az servicebus queue create -g rg-msg \
--namespace-name sb-contoso-001 -n orders \
--enable-duplicate-detection true \
--max-delivery-count 5
```

Geo-Disaster Recovery aliases replicate namespace metadata but not message data. Geo-Replication in supported Premium configurations replicates metadata and message data. Architecture documents must name the feature and its data semantics explicitly; otherwise teams may believe queued work is protected when it is not.

37.4 Operations and Reconciliation

Monitor active messages, incoming/outgoing rates, dead-letter count, scheduled messages, throttling, server errors, lock loss, processing latency, session backlog, and consumer health. Alert on age of oldest work and business completion, not only queue depth. A low queue can hide dropped sends; a high queue may be healthy during a planned batch.

- Assign each DLQ an owner, alert, retention policy, inspection tool, replay control, and sensitive-data handling process.
- Use tracing and correlation IDs across producer, broker, consumer, and business record.
- Test receiver outage, poison message, duplicate send, lock expiration, namespace failover, and replay.
- Reconcile source business records against completed outcomes to detect missing or duplicated effects.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Messages repeatedly return	Consumer fails, abandons, exceeds lock, or cannot complete	Inspect delivery count, lock-lost errors, processing duration, dependency failures, and renewal settings; make work shorter or checkpointed.
DLQ grows silently	No owner/alert, max delivery exceeded, TTL expired, or filter error	Inspect DLQ reason and description, establish alert and triage, correct consumer or contract before controlled replay.
Unauthorized after role assignment	Wrong identity, scope, token cache, or network path	Confirm runtime principal, role scope, token refresh, private DNS, and firewall. Avoid switching to connection strings as a workaround.
Failover loses queued work	Geo-DR alias was mistaken for data replication	Review configured feature and recovery test; use Geo-Replication where supported or design producer replay/reconciliation.

Hands-on Lab 37: Implement an Idempotent Topic Workflow with DLQ Operations

Prerequisites

- A sandbox Service Bus namespace and managed identities for producer and consumers.
- A small producer/consumer script or Functions application.
- A synthetic order payload with stable IDs.

Implementation

1. Create a topic with two filtered subscriptions representing independent business consumers.
2. Assign sender and receiver roles to separate managed identities.
3. Publish messages with stable MessageId, correlation ID, TTL, and business key.
4. Implement peek-lock consumers with idempotency records and bounded retry.
5. Force one poison message and prove it reaches the correct subscription DLQ with useful reason.
6. Build a controlled DLQ inspection and replay procedure that checks the underlying defect first.
7. Document whether the namespace uses no geo feature, Geo-DR, or Geo-Replication and the exact data protection provided.

Validation Evidence

- Each matching subscription processes independently.
- Duplicate delivery produces one business outcome per consumer.
- DLQ alert and evidence identify owner, reason, and correlation ID.
- Replay is authorized, bounded, and does not duplicate completed work.

Cleanup

- Delete test entities, messages, identities, role assignments, and namespace if not reused.
- Remove private endpoints and diagnostic workspaces created only for the lab.

Interview Questions and Model Answers

1. What is the difference between a queue and a topic?

A queue distributes work among competing receivers. A topic publishes each message to matching subscriptions, each with independent delivery state.

2. Why does duplicate detection not eliminate the need for idempotency?

It operates only within a configured time window and broker scope; consumer retries and business duplicates can still occur.

3. What is the Service Bus dead-letter queue?

A subqueue that retains messages that expired, exceeded delivery attempts, were explicitly dead-lettered, or failed certain subscription rules, pending owned triage.

4. How do Geo-DR and Geo-Replication differ?

Geo-DR preserves namespace configuration and alias continuity but not queued message data. Supported Geo-Replication replicates metadata and message data.

Chapter Summary

- Model business delivery semantics before creating entities.
- Use peek-lock, idempotency, bounded retry, and owned DLQs.
- Use managed identity, private networking, and capacity-aware concurrency.
- State geo-recovery data semantics explicitly and test them.

Chapter 38 - Azure Event Grid and Event Distribution

Routing reactive events with filtering, CloudEvents, push and pull delivery, dead-lettering, domains, namespaces, and contract governance

Learning Objectives

- Distinguish events from commands, streams, and brokered work.
- Design topics, domains, namespaces, subscriptions, filters, schemas, delivery, and dead-lettering.
- Implement secure publishers and handlers with managed identity and private networking where supported.
- Operate at-least-once delivery, replay, contract evolution, and event-driven observability.

38.1 Events Describe Facts; They Do Not Command Ownership

Azure Event Grid is a managed publish-subscribe event distribution service. An event states that something happened; consumers decide whether and how to react. Commands request a specific action and usually need stronger ownership and processing semantics. High-volume ordered telemetry streams are generally better served by Event Hubs, covered in Part C.

Need	Recommended Starting Point
Reactive notification to multiple handlers	Event Grid topic/system topic with filtered subscriptions.
Durable command or work item with broker controls	Service Bus queue or topic.
High-throughput ordered partitioned stream	Event Hubs.
Stateful integration sequence	Logic Apps, Durable Functions, or application workflow using events/messages.

38.2 Schemas, Filters, Delivery, and Dead-Lettering

Use CloudEvents where interoperability is valuable, or define a governed custom event schema. Include stable event type, version, source, subject, unique ID, occurrence time, correlation ID, tenant or partition context, and a payload that avoids unnecessary sensitive data. Consumers should tolerate additive fields and reject incompatible major versions clearly.

- Use subject, event type, advanced filters, and domains to route events without requiring every consumer to inspect every payload.
- Assume at-least-once delivery and nondeterministic ordering unless a specific design provides stronger behavior.
- Configure dead-letter storage for undeliverable events and assign ownership, retention, alerting, and replay procedures.
- Use retry and expiration settings from business validity. An old event may be technically deliverable but no longer safe to apply.

38.3 Push, Pull, Namespaces, and Security

Event Grid can deliver to Azure Functions, Logic Apps, Service Bus, webhooks, and other supported handlers. Namespace capabilities add broker-oriented event and MQTT scenarios and can support pull-style consumption patterns where applicable. Select the model from consumer availability, network posture, throughput, and control needs.

```
az eventgrid topic create -g rg-events -n orders-events \
  --location <region>
az eventgrid event-subscription create \
  --name accounting-created \
  --source-resource-id <topic-resource-id> \
  --endpoint-type servicebusqueue \
  --endpoint <queue-resource-id> \
  --included-event-types Contoso.OrderCreated.v1
```

Authenticate publishers through Entra and scoped roles. Validate webhook ownership and TLS. For Azure handlers, prefer managed identity and private networking options supported by the chosen resource model. Protect dead-letter storage and never treat it as an ungoverned payload archive.

38.4 Event Contracts and Operations

An event contract is a product. It requires an owner, schema repository, compatibility policy, example payloads, consumer inventory, privacy classification, deprecation process, and production metrics. Publishing an event before the originating transaction is committed can create phantom reactions; use transactional outbox or change-feed patterns when atomic publication matters.

- Monitor publish failures, matched events, delivery success, retry, dead-letter count, handler latency, and business completion.
- Propagate trace context or correlation identifiers through publisher, Event Grid, handler, and downstream work.
- Test unavailable handler, invalid event, duplicate event, expired event, filter change, and dead-letter replay.
- Use replay only with consumer idempotency and an approved event range to avoid a replay storm.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Subscription receives nothing	Source, event type, subject filter, endpoint, or authorization mismatch	Inspect topic metrics, subscription filters, event schema, endpoint validation, and handler logs; publish a known test event.
Webhook repeatedly retries	Handler is slow, unavailable, returns non-success, or TLS/DNS fails	Measure handler response, TLS chain, DNS, timeout, and response code; decouple with Service Bus if processing cannot acknowledge quickly.
Duplicate business result	Consumer assumed exactly-once delivery	Use event ID/business key idempotency, conditional state transition, and replay-safe processing.
Dead-letter replay causes spike	Unbounded range or original defect remains	Quarantine, fix handler, select a bounded event set, throttle replay, and reconcile outcomes.

Hands-on Lab 38: Publish a Governed Event and Route It to Two Handlers

Prerequisites

- An Event Grid topic or supported system topic.

- A Functions/Logic Apps handler and a Service Bus queue or second handler.
- A dead-letter storage account and synthetic event schema.

Implementation

1. Define an OrderCreated v1 CloudEvent with owner, compatibility rules, and data classification.
2. Create a topic and two filtered subscriptions for different consumers.
3. Configure dead-lettering and bounded retry/expiration.
4. Publish valid, duplicate, expired, and incompatible test events.
5. Implement consumer idempotency using event ID and business key.
6. Disable one handler, observe retry and dead-letter behavior, then restore it.
7. Replay a bounded dead-letter set and reconcile business outcomes.

Validation Evidence

- Only matching subscriptions receive each event.
- Duplicate delivery produces one result per consumer.
- Undeliverable events are protected, observable, and owned.
- Replay is bounded and traceable from source event to final outcome.

Cleanup

- Delete test subscriptions, topic, dead-letter blobs, handlers, identities, and role assignments.
- Remove stored payloads according to the lab data-retention rule.

Interview Questions and Model Answers

1. What is the difference between an event and a command?

An event records a fact that occurred and may have many consumers. A command asks a specific owner to perform an action and needs explicit processing semantics.

2. Why can Event Grid deliver duplicates?

It uses at-least-once delivery; retries after ambiguous acknowledgments or failures may redeliver an event.

3. When should Event Grid deliver to Service Bus instead of directly to a webhook?

When the consumer needs durable broker buffering, controlled concurrency, sessions, DLQ operations, or cannot acknowledge quickly and reliably.

4. What is transactional outbox used for?

It records the business change and event intent atomically in one data transaction, then publishes asynchronously so committed changes are not lost or falsely announced.

Chapter Summary

- Use Event Grid for reactive fact distribution, not every messaging need.
- Govern schemas, filters, retry, dead-lettering, and compatibility.
- Assume at-least-once delivery and build idempotent handlers.
- Operate event contracts and replay as production products.

Chapter 39 - Azure Logic Apps and Durable Workflow Engineering

Orchestrating integrations and long-running business processes with connectors, state, compensation, identity, observability, and controlled change

Learning Objectives

- Choose Logic Apps Consumption, Logic Apps Standard, Durable Functions, or application code from workflow requirements.
- Design triggers, actions, connectors, state, retries, timeouts, compensation, and human approval safely.
- Use managed identity, private networking, integration accounts, and secret controls.
- Troubleshoot trigger state, connector authentication, concurrency, run history, replay, and long-running failures.

39.1 Workflow Engine Selection

Azure Logic Apps is a low-code integration and workflow platform with managed and built-in connectors. Durable Functions provides code-first stateful orchestration. Application workflow libraries provide maximum code control but require more operational implementation. Selection depends on connector value, latency, throughput, networking, runtime control, versioning, and team ownership.

Option	Strength	Use Carefully When
Logic Apps Consumption	Per-action multitenant workflows, broad managed connectors, rapid integration	Strict private networking, very high steady throughput, or extensive source-control customization.
Logic Apps Standard	Single-tenant runtime, multiple workflows per resource, built-in connectors, stronger networking and deployment control	Plan capacity, runtime operations, connector differences, and local development.
Durable Functions	Code-first orchestration, activities, entities, deterministic replay	Team lacks code ownership or needs many SaaS connectors.
Application code	Full control and domain integration	Reimplementing durable state, retries, timers, monitoring, and operations.

39.2 State, Retries, Compensation, and Human Work

Long-running workflows cross systems that cannot share one transaction. Use saga-style steps with explicit compensation, reconciliation, and terminal states. A retry is appropriate for transient failure; compensation is required when a prior successful step must be reversed; manual review is required when automation cannot determine a safe outcome.

- Use idempotency keys for every external side effect and persist business state separately from UI status.
- Bound retries and timeouts. Route unhandled failures to an owned incident or exception queue.
- Model cancellation, expiry, partial success, and duplicate approval explicitly.
- Do not store sensitive payloads indefinitely in run history. Configure secure inputs/outputs and retention appropriate to the data.

39.3 Identity, Connectors, and Networking

Prefer managed identity for Azure services and use one connection per identity where connector behavior requires it. Separate design-time permissions from runtime permissions. Store unavoidable credentials in Key Vault and rotate them through controlled connection updates.

```
# Workflow design pseudocode
Trigger: Service Bus OrderRequested
1. Validate schema and idempotency key
2. Reserve inventory (retry transient failures)
3. Authorize payment
4. If payment fails: release inventory
5. Publish OrderAccepted or OrderRejected
6. Persist terminal state and correlation evidence
```

For Standard workflows, design VNet integration, private endpoints, DNS, storage access, and built-in connector behavior together. Managed connectors may use separate infrastructure paths and connection resources; verify the actual network and identity model for each connector.

39.4 Deployment, Observability, and Workflow Versioning

Treat workflow definitions, parameters, connections, schemas, maps, policies, and infrastructure as source-controlled artifacts. Deploy through rings and prevent a new definition from breaking in-flight instances. Durable Functions orchestrations require version-safe code; Logic Apps changes require awareness of trigger state and run history.

- Capture workflow/run ID, business ID, trigger ID, external request IDs, action duration, retry count, and terminal state.
- Alert on stuck duration, repeated action failure, exception queue growth, disabled triggers, and business reconciliation gaps.
- Test connector outage, duplicate trigger, timeout, compensation failure, human approval expiry, and deployment during in-flight work.
- Provide operators with safe resubmit/replay rules and make every manual action auditable.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Trigger is not firing	Disabled workflow, connection failure, polling state, filter, network, or authorization	Inspect trigger history, connection status, identity, recurrence/polling watermark, network path, and source metrics.
Workflow loops or duplicates actions	Duplicate trigger, retry, missing idempotency, or incorrect condition	Trace run IDs and business keys, inspect retry policy, add conditional state transitions and idempotent external calls.
Standard workflow cannot reach private service	VNet integration, DNS, storage, route, or connector path mismatch	Identify built-in versus managed connector, resolve DNS from runtime, inspect integration subnet and downstream firewall logs.

Symptom	Likely Cause	Evidence and Corrective Action
Compensation fails	Reverse action is not idempotent, dependency is unavailable, or original state changed	Persist compensation state, retry safely, escalate to manual reconciliation, and never mark the saga successful prematurely.

Hands-on Lab 39: Build a Compensating Order Workflow with Managed Identity

Prerequisites

- Logic Apps Standard or Consumption resource suitable to the sandbox.
- Service Bus queue/topic and two synthetic service endpoints.
- Managed identities and a protected telemetry destination.

Implementation

1. Model order states, idempotency key, timeout, compensation, and terminal outcomes before building the workflow.
2. Create a trigger from Service Bus or Event Grid using managed identity where supported.
3. Call an inventory endpoint and payment endpoint with bounded retry and correlation headers.
4. Implement compensation that releases inventory when payment fails.
5. Publish a terminal event and persist the workflow outcome.
6. Force duplicate trigger, transient failure, permanent payment failure, and compensation failure.
7. Create an operator runbook for safe resubmit or manual reconciliation.

Validation Evidence

- Duplicate triggers do not reserve or charge twice.
- Transient failure retries within policy; permanent failure follows compensation.
- Compensation failure is visible and reaches owned manual reconciliation.
- Run history and telemetry link the business ID to every external action.

Cleanup

- Delete workflows, connections, messages, test endpoints, identities, and role assignments.
- Remove run-history payloads or test data according to retention policy.

Interview Questions and Model Answers

1. When would you choose Logic Apps Standard over Consumption?

When single-tenant runtime, stronger networking, multiple workflows per resource, built-in connectors, source-control/deployment control, or predictable capacity is required.

2. What is a saga?

A sequence of local transactions coordinated by a workflow, with compensating actions or reconciliation when later steps fail.

3. Why is retry not the same as compensation?

Retry repeats an intended operation after transient failure. Compensation reverses or mitigates a prior successful operation when the overall process cannot complete.

4. How do you make workflow resubmission safe?

Use idempotency keys, persisted state, bounded event selection, authorization, audit evidence, and checks that the original defect is fixed before resubmitting.

Chapter Summary

- Select workflow technology from connector, runtime, networking, and ownership needs.
- Model state, idempotency, retries, compensation, expiry, and manual reconciliation explicitly.
- Use managed identity and verify each connector network path.
- Treat workflow definitions and replay operations as governed production code.

Chapter 40 - Enterprise Cloud-Native Application Architecture and Capstone

Integrating ACR, AKS, Container Apps, App Service, Functions, APIM, Service Bus, Event Grid, Logic Apps, identity, networking, observability, delivery, and recovery

Learning Objectives

- Compose Azure application platforms from workload semantics rather than service popularity.
- Design end-to-end identity, network, API, event, message, data, delivery, and observability flows.
- Run failure simulations, recovery exercises, security validation, and cost review.
- Produce a portfolio-ready architecture, runbook set, evidence pack, and executive trade-off summary.

40.1 Capstone Scenario: Global Order and Fulfillment Platform

Contoso Commerce is replacing a tightly coupled regional application with a cloud-native platform. Customers submit orders through public APIs. Inventory, payment, fulfillment, notification, and analytics evolve independently. The system must support controlled global delivery, private backend access, duplicate-safe processing, burst absorption, auditable workflows, regional recovery, and least-privilege identities.

Capability	Reference Azure Component	Reason
Artifact trust	ACR Premium with private access and digest promotion	Controlled supply chain, regional distribution, runtime pull identity.
Public API edge	Front Door/WAF plus API Management	Global entry, TLS/WAF, API contract, token, quota, routing, and analytics.
Synchronous services	App Service, Container Apps, or AKS by requirement	Use the lowest platform complexity that satisfies runtime and ecosystem needs.
Commands and work	Service Bus queues/topics	Durable buffering, controlled concurrency, sessions, DLQ, and replay.
Reactive facts	Event Grid	Filtered fan-out of business and platform events.
Short event logic	Azure Functions	Elastic handlers with idempotency and managed identity.
Long process	Logic Apps Standard or Durable Functions	State, timers, compensation, and operator visibility.

40.2 Reference Flow and Trust Boundaries

1. Client resolves the global endpoint and connects through WAF-protected TLS.
2. API Management validates the token, applies consumer quota, adds trace context, and calls the order API using managed identity or mTLS as designed.
3. The API writes the order and an outbox record atomically, then returns a durable order ID.
4. An outbox publisher sends an OrderRequested command to Service Bus with MessageId and correlation ID.

5. An orchestrator reserves inventory and authorizes payment using idempotent activities; failure triggers compensation.
6. OrderAccepted or OrderRejected is published as a governed event through Event Grid.
7. Independent notification, analytics, and fulfillment consumers react without changing the order API.
8. Every component emits structured telemetry linked by trace ID, event ID, message ID, workflow ID, and order ID.

Use private endpoints and private DNS for ACR, messaging, secrets, storage, and data services where requirements justify them. Public edge components are explicitly separated from internal application and data subnets. Each workload has a dedicated managed identity or Workload ID and only the permissions needed for its role.

40.3 Delivery, Security, Reliability, and Cost Gates

Gate	Required Evidence
Supply chain	Reviewed source, scans, SBOM, provenance, approved digest, ACR promotion, runtime admission evidence.
Identity	Identity inventory, role-scope map, no embedded secrets, privileged-access procedure, token and key rotation test.
Network	Data-flow diagram, public/private endpoint inventory, DNS resolution tests, firewall/NSG evidence, egress path.
Reliability	SLOs, capacity assumptions, retries, timeouts, idempotency, DLQ ownership, compensation, recovery test.
Observability	Dashboards, alerts, trace correlation, deployment markers, synthetic tests, business reconciliation.
Cost	Tagged forecast, scale floors/ceilings, log retention, premium-service justification, cleanup and anomaly alert.

Release through environments using the same immutable artifact. Canary the API or container revision, compare technical and business telemetry, and promote only after approval. Database and event contract changes remain backward compatible through the rollout window.

40.4 Failure Simulations and Recovery

- Deny ACR access for new pulls while keeping existing workloads running; verify deployment failure is visible and rollback artifacts remain available.
- Stop a consumer and grow Service Bus backlog; prove safe scale-out, dependency protection, and oldest-message alerts.
- Inject duplicate messages and events; prove one business outcome and complete trace evidence.
- Break a private DNS record; prove diagnosis from resolution to application symptom without disabling network controls.
- Fail payment after inventory reservation; prove compensation and manual reconciliation when compensation also fails.
- Simulate a regional application outage; execute traffic failover and prove dependencies, messaging strategy, data recovery, and operator communications.

Recovery success is not the return of HTTP 200 alone. It includes data correctness, message/event reconciliation, restored observability, security controls, capacity, customer communication, and documented evidence that objectives were met.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Order API is healthy but orders stop progressing	Outbox publisher, Service Bus consumer, workflow, or dependency is stalled	Trace order ID through outbox, send, queue, receive, workflow, and terminal event; inspect oldest age and DLQ before replay.
Canary has low errors but business conversion drops	Technical health misses semantic regression	Compare business KPIs, contract behavior, downstream events, and feature flags; roll back the digest and preserve evidence.
Regional failover produces authorization failures	Secondary identities, roles, DNS, certificates, or private endpoints are incomplete	Validate every regional dependency and identity before failover; use automated conformance tests and correct infrastructure code.
Cost spikes after incident	Scale ceiling absent, retry storm, verbose logs, or Premium resources left expanded	Correlate cost with replicas, requests, retries, log ingestion, and temporary capacity; cap safely and record incident cost action.

Hands-on Lab 40: Build and Defend the Enterprise Cloud-Native Capstone

Prerequisites

- Completion of Parts A-D labs or equivalent Azure knowledge.
- A sandbox subscription with strict budget and a reduced-cost architecture.
- Repository for IaC, application code, API/event schemas, tests, runbooks, and evidence.

Implementation

1. Write functional and nonfunctional requirements, SLOs, RPO/RTO, data classification, budget, and architecture decision records.
2. Create the landing resource groups, managed identities, network/DNS plan, ACR, telemetry, and secrets controls through IaC.
3. Deploy one synchronous API, one queue/topic, one event route, one function/worker, and one compensating workflow.
4. Publish the API through APIM with token validation, quota, correlation, and standardized errors.
5. Build once, promote an image by digest, and perform a canary release with rollback.
6. Run duplicate, poison, dependency outage, DNS failure, and consumer backlog tests.
7. Execute a reduced regional recovery or rebuild exercise and reconcile all business records.
8. Produce an architecture diagram, threat model, runbooks, dashboards, evidence pack, cost report, and executive trade-off summary.

Validation Evidence

- Every deployed artifact is traceable to source, scan, digest, approval, and environment.
- Every runtime uses managed identity or Workload ID with least privilege.
- Duplicate and replay tests preserve one business outcome.
- DLQ, compensation, and manual reconciliation have named owners and tested procedures.

- Failure tests meet or honestly measure the defined SLO/RPO/RTO.
- The environment can be rebuilt or cleaned without undocumented manual configuration.

Cleanup

- Export required evidence, remove synthetic data, revoke temporary permissions, and destroy all capstone resources through IaC.
- Verify budgets, cost analysis, orphaned disks/IPs/private endpoints, Log Analytics retention, APIM/AKS/plan capacity, and ACR replicas.

Interview Questions and Model Answers

1. How do you decide between AKS, Container Apps, App Service, and Functions?

Start from runtime control, Kubernetes ecosystem, container need, event model, scaling, networking, latency, portability, team skill, and operational cost; choose the least complex platform that meets them.

2. Why use both Service Bus and Event Grid in one architecture?

Service Bus handles owned durable commands and work with broker controls. Event Grid distributes facts reactively to independent consumers.

3. What is the most important evidence in a cloud-native release?

Traceability from reviewed source to immutable artifact and deployed digest, plus policy results, approval, telemetry, rollback target, and business validation.

4. How do you prove regional resilience?

Execute a realistic failover or rebuild that validates traffic, identity, DNS, dependencies, messages/events, data correctness, observability, capacity, and operator runbooks against measured objectives.

Chapter Summary

- Compose platforms from workload semantics and operational ownership.
- Connect artifact trust, identity, networking, delivery, messaging, events, workflow, and observability end to end.
- Use canaries, idempotency, DLQs, compensation, reconciliation, and tested recovery.
- Deliver architecture and operational evidence suitable for a professional portfolio and production review.

Part D Command Reference

Commands are examples for a sandbox. Validate current CLI extension versions, service availability, names, quotas, and organization policy before use. Replace placeholders and do not paste secrets into shell history.

Area	Useful Inspection Commands
ACR	<pre>az acr show -g <rg> -n <registry> az acr repository show-manifests -n <registry> --repository <repo></pre>
AKS	<pre>az aks get-credentials -g <rg> -n <cluster> kubectl get pods -A -o wide kubectl get events -A --sort-by=.lastTimestamp</pre>
Container Apps	<pre>az containerapp revision list -g <rg> -n <app> -o table az containerapp logs show -g <rg> -n <app> --follow</pre>
App Service	<pre>az webapp log tail -g <rg> -n <app> az webapp deployment slot list -g <rg> -n <app> -o table</pre>
Functions	<pre>az functionapp show -g <rg> -n <app> az monitor app-insights query --app <app-id> --analytics-query <query></pre>
API Management	<pre>az apim show -g <rg> -n <apim> az apim api list -g <rg> --service-name <apim> -o table</pre>
Service Bus	<pre>az servicebus queue show -g <rg> --namespace-name <ns> -n <queue> az servicebus topic subscription list -g <rg> --namespace-name <ns> --topic-name <topic></pre>
Event Grid	<pre>az eventgrid event-subscription list --source-resource-id <id> -o table</pre>
Resource cleanup	<pre>az resource list -g <rg> -o table az group delete -n <rg> --yes --no-wait</pre>

Production-Readiness Checklist

- Artifact is immutable by digest, scanned, traceable, and retained for rollback.
- Build, deployment, and runtime identities are separate and least privileged.
- Public and private endpoints, DNS, routes, firewalls, and egress are documented and tested.
- Applications have requests/limits or equivalent capacity settings, health probes, timeouts, and graceful shutdown.
- Autoscaling has meaningful signals, minimums, maximums, dependency protection, and cost limits.
- Every queue, subscription, and event route has retry, expiry, idempotency, dead-letter, alert, and owner.
- API authentication, authorization, versioning, quota, certificate, error contract, and deprecation are tested.
- Workflow compensation, manual reconciliation, and safe replay are documented and exercised.
- Telemetry correlates source request/event/message to business outcome and deployment version.
- Regional recovery or rebuild validates identity, DNS, network, data, messaging, observability, and capacity.
- Cost forecast, budget, log retention, premium feature justification, and cleanup automation are approved.
- Runbooks name decision authority, escalation, rollback, replay, and customer-communication actions.

Cross-Service Troubleshooting Matrix

Symptom	Cross-Service Investigation
Deployment succeeds but runtime fails	Verify deployed digest, runtime identity, ACR pull, configuration, private DNS, dependency roles, firewall, probes, and application telemetry.
Requests are slow end to end	Split edge, APIM, ingress, application, queue wait, workflow, and dependency latency using trace and business correlation.
Duplicate business results	Trace request/event/message/workflow IDs; inspect retry and replay paths; enforce conditional state transitions and idempotency records.
Private service is unreachable	Resolve DNS from the actual runtime, inspect route and NSG/firewall flow, private endpoint approval, and application egress integration.
Backlog grows after release	Compare consumer version, errors, processing time, concurrency, lock loss, dependency throttling, and scale ceiling; roll back if semantic regression.
Failover is partly successful	Validate identity/roles, DNS, certificates, private endpoints, data recovery, broker geo semantics, telemetry, and hidden regional dependencies.
Cost rises unexpectedly	Correlate replicas, nodes, plans, APIM/Service Bus tiers, ACR replicas, retries, log ingestion, and abandoned private resources.

Glossary

Term	Definition
ACR	Azure Container Registry, a private registry for container images and OCI artifacts.
AKS	Azure Kubernetes Service, a managed Kubernetes control-plane service.
OCI artifact	An artifact stored using Open Container Initiative distribution specifications, including images and other package types.
Digest	A content-derived immutable identifier for an image or artifact.
Revision	An immutable version snapshot used by Container Apps or API lifecycle features according to service semantics.
Workload ID	Federated Microsoft Entra authentication that lets a Kubernetes service account obtain Azure tokens without stored secrets.

Term	Definition
KEDA	Kubernetes Event-driven Autoscaling, used by AKS add-ons and Container Apps scaling.
HPA	Horizontal Pod Autoscaler, which changes pod replicas from metrics.
Deployment slot	A live App Service deployment environment that can be validated and swapped with production.
Cold start	Latency incurred while a serverless runtime or instance is allocated and initialized.
Idempotency	Property that repeated processing of the same operation produces one intended business effect.
APIM	Azure API Management, including gateway, policies, lifecycle, products, and developer capabilities.
Peek-lock	Service Bus receive mode that locks a message until the receiver completes, abandons, defers, or dead-letters it.
DLQ	Dead-letter queue or destination for work that cannot be processed normally.
CloudEvents	A vendor-neutral event-envelope specification.
Saga	A long-running business process implemented as local transactions with compensation or reconciliation.
Outbox	A pattern that stores business change and event intent atomically before asynchronous publication.
RPO	Recovery point objective, the acceptable amount of data loss measured in time.
RTO	Recovery time objective, the target time to restore service.
SLO	Service level objective, a measurable reliability target for a service.

Official Microsoft Documentation Index

Reference	Microsoft Learn URL
Azure Container Registry documentation	https://learn.microsoft.com/azure/container-registry/
Azure Kubernetes Service documentation	https://learn.microsoft.com/azure/aks/
AKS Workload ID	https://learn.microsoft.com/azure/aks/workload-identity-overview
Azure Container Apps documentation	https://learn.microsoft.com/azure/container-apps/
Azure App Service documentation	https://learn.microsoft.com/azure/app-service/
Azure Functions documentation	https://learn.microsoft.com/azure/azure-functions/

Reference	Microsoft Learn URL
Functions Flex Consumption	https://learn.microsoft.com/azure/azure-functions/flex-consumption-plan
Azure API Management documentation	https://learn.microsoft.com/azure/api-management/
Azure Service Bus documentation	https://learn.microsoft.com/azure/service-bus-messaging/
Azure Event Grid documentation	https://learn.microsoft.com/azure/event-grid/
Azure Logic Apps documentation	https://learn.microsoft.com/azure/logic-apps/
Azure Architecture Center	https://learn.microsoft.com/azure/architecture/
Azure Well-Architected Framework	https://learn.microsoft.com/azure/well-architected/

Service capabilities, regions, tiers, limits, pricing, and lifecycle status change. Revalidate them against current Microsoft documentation and organizational policy before production implementation.